Часть 2. Массивы, строки, методы

2.2. Многомерные и ступенчатые массивы

К концу занятия ты сможешь:

- хранить таблицу или игровое поле в одной переменной
- обходить таблицу вложенными циклами
- считать суммы по строкам и по столбцам
- хранить строки разной длины в ступенчатом массиве

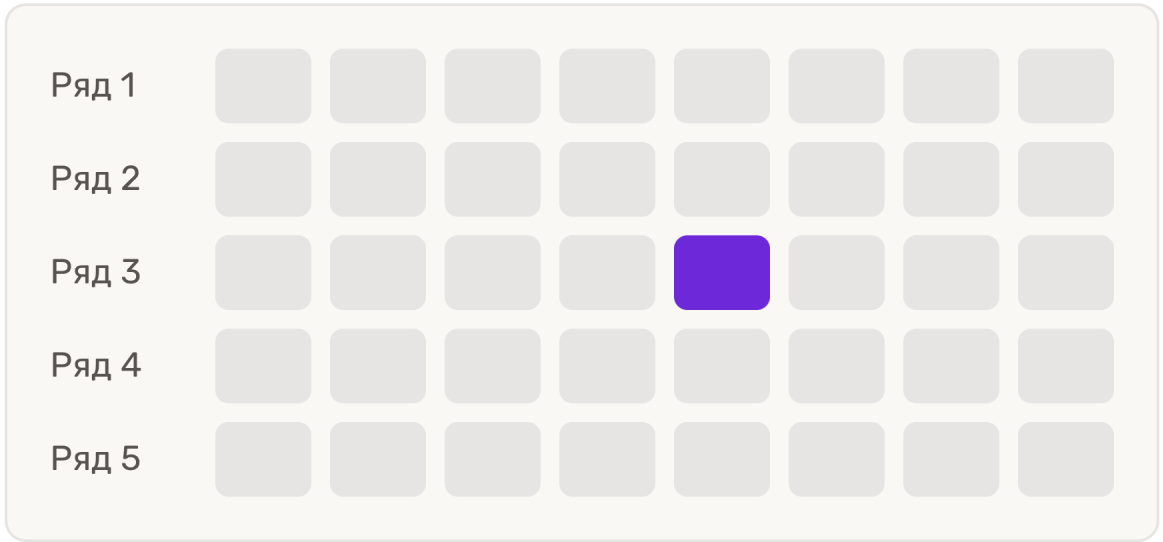
Разминка

Program.cs

```
int[] lives = { 3, 2, 5, 1 };
```

1. Какой индекс у последнего элемента? Что выведет lives[^1]?
2. Почему lives[4] вызывает ошибку?
3. Почему внутри foreach (int item in lives) нельзя написать item = 0, чтобы обнулить элемент?

Кинозал на 40 мест



Экран — сверху. Выделено: ряд 3, место 5

Как хранить 40 мест и сразу находить «ряд 3, место 5»?

В обычном массиве из 40 элементов придётся считать индекс:

```
формула
(ряд - 1) * 8 + место - 1   // = 20
```

Ошибся на единицу — забронировал чужое место.

К концу урока ты запишешь это так: hall[2, 4]

План занятия

Урок 1

- Разминка
- Двумерный массив: создание, ячейки, размеры
- Перебор вложенными циклами, суммы
- Живой кодинг «Карта сокровищ»
- Практика «Журнал оценок»

Урок 2

- Ступенчатые массивы: строки разной длины
- Живой кодинг «Длительность плейлистов»
- Практика «Дневник пробежек»
- Мини-проект «Кинозал-амфитеатр»
- Итоги

Вспомним: вложенные циклы (1.5) и массив (2.1)

Program.cs

```
for (int row = 0; row < 3; row++)
{
    for (int col = 0; col < 5; col++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
```

Вывод в консоли

```
*****
*****
*****
```

- Внешний цикл — строки, внутренний — символы в строке.
- Внутренний цикл проходит целиком за один шаг внешнего.
- Индексы массива начинаются с 0, последний — Length - 1.

Блок 1 из 2

Двумерный массив — таблица

В этом блоке ты научишься:

- создавать таблицу в одной переменной
- читать и менять ячейку по номеру строки и столбца
- узнавать размеры таблицы
- обходить таблицу вложенными циклами

Что такое двумерный массив

Двумерный массив — таблица из ячеек одного типа. У каждой ячейки **два номера**: строка и столбец.

Как место в кино: «ряд 3, место 5». Или клетка шахматной доски.

Зачем

Без него: 40 мест в обычном массиве и формула индекса, в которой легко ошибиться.

С ним: hall[ряд, место] — пишешь так, как думаешь.

	столбец 0	**столбец 1**	**столбец 2**	**столбец 3**
строка 0				
строка 1			ячейка [1, 2]	
строка 2				

Как создать таблицу

Program.cs

```
int[,] field = new int[3, 4];
```

int[,] — тип «таблица из int». Запятая внутри скобок = два номера

3 — сколько строк

4 — сколько столбцов

	0	1	2	3
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0

После создания все ячейки равны 0, как и в обычном массиве.

Правило: сначала строка, потом столбец.

Читаем и меняем ячейку

Program.cs

```
int[,] field = new int[3, 4];   // поле 3 × 4
field[1, 2] = 5;                // монета в строке 1, столбце 2
field[0, 3] = field[1, 2] * 2;  // монета ценой 10
Console.WriteLine(field[1, 2]);
Console.WriteLine(field[0, 3]);
Console.WriteLine(field[2, 0]);
```

	0	1	2	3
0	0	0	0	10
1	0	0	5	0
2	0	0	0	0

Вывод в консоли

```
5
10
0
```

Пустая ячейка [2, 0] не трогали — там 0.

Номера в скобках пишутся через запятую.

Таблица с готовыми значениями

Program.cs — очки по раундам

```
int[,] scores =
{
    { 12, 7, 9 },    // строка 0: Аня
    { 5, 11, 8 }     // строка 1: Борис
};
Console.WriteLine(scores[0, 1]);
Console.WriteLine(scores[1, 2]);
```

Вывод в консоли

```
7
8
```

	раунд 0	раунд 1	раунд 2
0: Аня	12	7	9
1: Борис	5	11	8

Каждая строка — в своих фигурных скобках, строки через запятую.

Все строки должны быть **одной длины**.
Иначе ошибка:

```
CS0847: An array initializer of length '3'
is expected
```

«Ожидается инициализатор массива длины 3»

Строки разной длины — во втором уроке.

Узнаём размеры: GetLength и Length

Program.cs

```
int[,] scores =
{
    { 12, 7, 9 },
    { 5, 11, 8 }
};
Console.WriteLine(scores.GetLength(0)); // строк
Console.WriteLine(scores.GetLength(1)); // столбцов
Console.WriteLine(scores.Length);       // всех ячеек
```

Вывод в консоли

```
2
3
6
```

Запись	Что даёт
GetLength(0)	число строк
GetLength(1)	число столбцов
Length	все ячейки: 2 × 3

Обходим таблицу вложенными циклами

Program.cs

```
int[,] scores = { { 12, 7, 9 }, { 5, 11, 8 } };
for (int row = 0; row < scores.GetLength(0); row++)
{
    for (int col = 0; col < scores.GetLength(1); col++)
    {
        Console.Write(scores[row, col] + "\t");
    }
    Console.WriteLine();
}
```

Вывод в консоли

```
12      7       9
5       11      8
```

"\t" — табуляция: ровные столбцы.

Порядок обхода:

[0,0] → [0,1] → [0,2] → новая строка → [1,0] → [1,1] → [1,2]

foreach тоже пройдёт все ячейки, но не скажет номер строки и столбца.

Сумма по строкам: очки каждого игрока

Program.cs

```
int[,] scores = { { 12, 7, 9 }, { 5, 11, 8 } };
for (int row = 0; row < scores.GetLength(0); row++)
{
    int sum = 0;  // новая сумма для каждого игрока
    for (int col = 0; col < scores.GetLength(1); col++)
    {
        sum += scores[row, col];
    }
    Console.WriteLine($"Игрок {row}: {sum}");
}
```

Вывод в консоли

```
Игрок 0: 28
Игрок 1: 24
```

row	sum по шагам
0	0 → 12 → 19 → 28
1	0 → 5 → 16 → 24

Правило: sum = 0 — внутри внешнего цикла, перед внутренним.

Сумма по столбцам: очки каждого раунда

Program.cs

```
int[,] scores = { { 12, 7, 9 }, { 5, 11, 8 } };
for (int col = 0; col < scores.GetLength(1); col++)
{
    int sum = 0;  // сумма раунда
    for (int row = 0; row < scores.GetLength(0); row++)
    {
        sum += scores[row, col];
    }
    Console.WriteLine($"Раунд {col}: {sum}");
}
```

Вывод в консоли

```
Раунд 0: 17
Раунд 1: 18
Раунд 2: 17
```

Что поменялось

Внешний цикл теперь идёт по **столбцам** (GetLength(1)), внутренний — по строкам.

Ячейка всё равно пишется [row, col]: сначала строка.

Ловушка: перепутали строки и столбцы

Program.cs

```
int[,] scores = { { 12, 7, 9 }, { 5, 11, 8 } };
// ошибка: строки считаем через GetLength(1)
for (int row = 0; row < scores.GetLength(1); row++)
{
    for (int col = 0; col < scores.GetLength(0); col++)
    {
        Console.Write(scores[row, col] + " ");
    }
    Console.WriteLine();
}
```

Вывод в консоли

```
12 7
5 11
Unhandled exception. System.IndexOutOfRangeException:
Index was outside the bounds of the array.
   at Program.<Main>$(String[] args) in Program.cs:line 7
```

«Индекс вне границ массива», строка 7.

Таблица 2 × 3, а цикл по строкам идёт до 3. Строки с номером 2 нет — программа падает.

Как правильно

строки — GetLength(0)

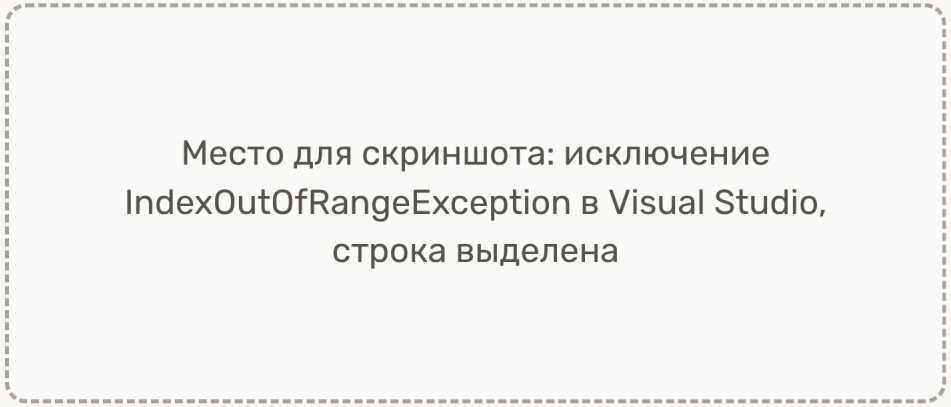
столбцы — GetLength(1)

в условии <, а не <=

На квадратной таблице ошибку не видно. Проверяй на прямоугольной.

Живой кодинг: «Карта сокровищ»

1. Создать карту `char[,] map = new char[4, 6];` и заполнить точками
2. Спрятать клад: `Random` выбирает строку 0–3 и столбец 0–5
3. Спросить у игрока строку и столбец
4. Отметить клетку: `$` — клад найден, `X` — мимо
5. Вывести карту вложенными циклами
6. Прочитать сообщение об ошибке, найти строку, исправить



Повторяй за преподавателем у себя в Visual Studio.

Вопросы по блоку 1

1. Что выведут `board.GetLength(1)` и `board.Length`, если `int[,] board = new int[2, 5];`?
2. Почему `field[3, 0]` вызывает ошибку, если `field = new int[3, 4]`?
3. В сумме по строкам строку `int sum = 0;` перенесли выше внешнего цикла. Что выведет программа для `{ { 12, 7, 9 }, { 5, 11, 8 } }`?
4. Найди ошибку: `for (int col = 0; col <= scores.GetLength(1); col++)`

Задание: «Журнал оценок»

Оценки 4 учеников по 5 предметам уже записаны в коде.

1. Выведи таблицу оценок ровными столбцами с названиями предметов.
2. Для каждого ученика выведи средний балл с одним знаком после запятой.
3. Для каждого предмета выведи средний балл.

Подсказка: среднее — (double)sum / количество, формат {avg:F1} (1.2). Количество бери из GetLength.

★ **Успел раньше?** В конце выведи, у кого лучший средний балл.

Застрял — 15 минут ищешь сам, потом спрашиваешь: что делал, что ожидал, что получил.

Program.cs — начальные данные

```
string[] students = { "Аня", "Борис", "Вика", "Глеб" };
string[] subjects = { "Мат", "Рус", "Физ", "Инф", "Анг" };
int[,] marks =
{
    { 5, 4, 5, 5, 4 },
    { 3, 4, 4, 5, 3 },
    { 5, 5, 4, 5, 5 },
    { 5, 3, 3, 3, 2 }
};
```

Что должно получиться

Вывод в консоли

```
        Мат     Рус     Физ     Инф     Анг
Аня     5       4       5       5       4
Борис   3       4       4       5       3
Вика    5       5       4       5       5
Глеб    5       3       3       3       2
Средний балл учеников:
Аня: 4,6
Борис: 3,8
Вика: 4,8
Глеб: 3,2
Средний балл по предметам:
Мат: 4,5
Рус: 4,0
Физ: 4,0
Инф: 4,5
Анг: 3,5
```

Проверь себя

- У Вики 4,8, у Глеба 3,2 — дробная часть не потерялась.
- По физике 4,0: сумма считается по столбцу, а не по строке.
- В циклах GetLength(0) и GetLength(1), а не числа 4 и 5.

★★ Для сильных: «Крестики-нолики: кто победил?»

Поле char[3, 3] уже заполнено. Программа проверяет 3 строки, 3 столбца и 2 диагонали и выводит Победил X, Победил O или Победителя нет.

Придумай, как проверить все строки одним циклом, а не тремя отдельными условиями.

Вывод в консоли

```
Победил X
```

Program.cs — начальные данные

```
char[,] board =
{
    { 'X', 'O', 'X' },
    { 'O', 'X', 'O' },
    { 'O', 'X', 'X' }
};
```

Блок 2 из 2

Ступенчатые массивы — строки разной длины

В этом блоке ты научишься:

- хранить строки разной длины в одной переменной
- узнавать длину каждой строки
- выбирать между int[,] и int[][]

Когда строки разной длины

Плейлисты: в одном 3 трека, в другом 5, в третьем 2. Длина трека — в секундах.

Для учёбы	185	210	174		
В дорогу	201	156	240	199	222
Спорт	130	175			

В int[,] пришлось бы сделать таблицу 3 × 5 и заполнить пустые места нулями. Потом не отличишь «пусто» от трека длиной 0.

Ступенчатый массив

Массив, в котором каждый элемент — сам массив. Поэтому каждая строка может быть своей длины.

Как полки разной длины в шкафу или ряды в амфитеатре: ближе к сцене — короче.

Как создать ступенчатый массив

Program.cs

```
int[][] playlists = new int[3][];                    // 3 строки, их пока нет
playlists[0] = new int[] { 185, 210, 174 };          // «Для учёбы»: 3 трека
playlists[1] = new int[] { 201, 156, 240, 199, 222 }; // «В дорогу»: 5 треков
playlists[2] = new int[] { 130, 175 };               // «Спорт»: 2 трека
Console.WriteLine(playlists[1][3]);                  // строка 1, элемент 3
```

Вывод в консоли

```
199
```

int[][] — массив массивов: две пары скобок

new int[3][] — 3 строки, вторые скобки пустые

[1][3] — сначала строка, потом элемент в ней

Обходим строки разной длины

Program.cs

```
int[][] playlists =
{
    new int[] { 185, 210, 174 },
    new int[] { 201, 156, 240, 199, 222 },
    new int[] { 130, 175 }
};
for (int i = 0; i < playlists.Length; i++)
{
    Console.Write($"Плейлист {i} ({playlists[i].Length} тр.): ");
    for (int j = 0; j < playlists[i].Length; j++)
    {
        Console.Write(playlists[i][j] + " ");
    }
    Console.WriteLine();
}
```

playlists.Length — сколько **строк** (3)

playlists[i].Length — сколько элементов в **строке i**

У каждой строки своя длина, поэтому внутренний цикл спрашивает её заново на каждом шаге.

Результат обхода

Вывод в консоли

```
Плейлист 0 (3 тр.): 185 210 174
Плейлист 1 (5 тр.): 201 156 240 199 222
Плейлист 2 (2 тр.): 130 175
```

i	playlists[i].Length	j проходит
0	3	0, 1, 2
1	5	0, 1, 2, 3, 4
2	2	0, 1

Правило: во внутреннем цикле граница — длина **текущей строки** [i].Length, а не число строк.

Ловушка: строку забыли создать

Program.cs

```
int[][] playlists = new int[3][];
playlists[0] = new int[] { 185, 210, 174 };
playlists[2] = new int[] { 130, 175 };
Console.WriteLine(playlists[1].Length);
```

Вывод в консоли

```
Unhandled exception. System.NullReferenceException:
Object reference not set to an instance of an object.
   at Program.<Main>$(String[] args) in Program.cs:line 4
```

«Ссылка на объект не указывает на экземпляр объекта» — строка 4.

new int[3][] создаёт только 3 пустых места для строк. Строку 1 никто не создал — узнать её длину нельзя.

Как правильно

Создай каждую строку через new int[...] до того, как к ней обращаться.

Пока просто запомни: «строки ещё нет» в C# называется null. Подробно — в теме 4.3.

int[,] или int[][]

	int[,] — таблица	int[][] — ступенчатый
Форма	все строки одной длины	строки разной длины
Создание	new int[3, 4]	new int[3][], затем каждая строка
Ячейка	a[1, 2]	a[1][2]
Число строк	a.GetLength(0)	a.Length
Длина строки	a.GetLength(1)	a[i].Length
Примеры	игровое поле, журнал, зал	плейлисты, тренировки, амфитеатр

Перепутал скобки?

field[1][2] для int[,]:

```
CS0022: Wrong number of indices inside []; expected 2
```

«Неверное количество индексов в [], ожидалось 2»

Живой кодинг: «Длительность плейлистов»

1. Названия плейлистов — в string[] names, длительности в секундах — в int[][] playlists
2. Для каждого плейлиста посчитать сумму вложенным циклом
3. Перевести секунды в минуты и секунды: / 60 и % 60 (1.3)
4. Запустить, прочитать ошибку, исправить границу цикла

Вывод в консоли — после исправления

```
Для учёбы: 9 мин 29 с
В дорогу: 16 мин 58 с
Спорт: 5 мин 5 с
```

Место для скриншота: исключение IndexOutOfRangeException во внутреннем цикле

Повторяй за преподавателем у себя в Visual Studio.

Вопросы по блоку 2

1. Для playlists из примера: чему равны playlists.Length и playlists[2].Length?
2. Почему у int[][] нельзя одним вызовом узнать «число столбцов», как GetLength(1) у int[,]?
3. Что произойдёт, если забыть создать строку 1 и обойти массив вложенным циклом?

Задание: «Дневник пробежек»

Километры пробежек за 4 дня уже записаны в коде. В день бывает от 1 до 3 пробежек.

1. Для каждого дня выведи все пробежки и сумму за день.
2. В конце выведи день с самым большим пробегом.

Подсказка: внутренний цикл — до runs[i].Length. Максимум ищи так же, как в теме 2.1.

★ **Успел раньше?** Выведи ещё среднюю длину пробежки за каждый день (один знак после запятой).

Program.cs — начальные данные

```
string[] days = { "Пн", "Вт", "Ср", "Чт" };
double[][] runs =
{
    new double[] { 3.5, 2 },
    new double[] { 5 },
    new double[] { 2.5, 1.5, 3 },
    new double[] { 4, 2.5 }
};
```

Что должно получиться

Вывод в консоли

```
Пн: 3,5 2 | всего 5,5 км
Вт: 5 | всего 5 км
Ср: 2,5 1,5 3 | всего 7 км
Чт: 4 2,5 | всего 6,5 км
Больше всего: Ср, 7 км
```

Проверь себя

- Во вторник одна пробежка — программа не падает.
- Внутренний цикл идёт до runs[i].Length, а не до 3.
- Добавь пятый день — программа работает без других правок.

★★ Для сильных: «Треугольник Паскаля»

Заполни ступенчатый массив: строка `i` содержит `i + 1` чисел. Крайние числа — 1, каждое внутреннее — сумма двух чисел над ним.

Выведи первые 8 строк.

Придумай сам, как взять «два числа над ним» из предыдущей строки.

Вывод в консоли

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
```

Вопросы по занятию

1. Чем int[,] похож на обычный массив из темы 2.1 и чем отличается?
2. Выбор между int[,] и int[][] похож на выбор между массивом и List (2.1). Почему?
3. В амфитеатре ряды по 6, 8 и 10 мест. Что будет, если проверять номер места по длине первого ряда hall[0].Length, а не hall[row].Length?

Мини-проект: «Кинозал-амфитеатр»

Ряды разной длины: 6, 8, 10 и 10 мест. Программа показывает зал, бронирует места и не даёт занять место дважды.

Вывод в консоли

```
Ряд 1: . . . . . .
Ряд 2: . . . . . . . .
Ряд 3: . . . . . . . . . .
Ряд 4: . . . . . . . . . .
Ряд (0 — выход): 2   <- ввёл пользователь
Место: 3   <- ввёл пользователь
Место забронировано.
Ряд 1: . . . . . .
Ряд 2: . . X . . . . .
Ряд 3: . . . . . . . . . .
Ряд 4: . . . . . . . . . .
```

Стрелка — пометка для тебя, в программе её нет.

Вывод в консоли — продолжение

```
Ряд (0 — выход): 2   <- ввёл пользователь
Место: 3   <- ввёл пользователь
Место занято, выбери другое.
Ряд (0 — выход): 1   <- ввёл пользователь
Место: 7   <- ввёл пользователь
Такого места нет.
Ряд (0 — выход): 0   <- ввёл пользователь
Хорошего сеанса!
```

Стрелка — пометка для тебя, в программе её нет.

Зал выводится при запуске и после каждой удачной брони.

Что должна уметь программа

- Показывать зал: в каждом ряду своё число мест, занятые места — X.
- Бронировать место по номеру ряда и места. Номера — с 1, как на билете в кино.
- Не давать занять одно место дважды.
- Отвечать «Такого места нет» на несуществующий ряд или место и на буквы вместо числа. Выходить по 0.

План по шагам

Шаг	Что сделать	Тема	Как проверить
1	Создать зал char[][] hall: ряды 6, 8, 10, 10 мест, заполнить точками	2.2	Запусти — ошибок нет
2	Вывести зал: «Ряд N:» и места через пробел	1.5, 2.2	4 ряда разной длины
3	Спросить ряд и место (int.TryParse), проверить по длине этого ряда, поставить X или сказать «занято»	1.2, 1.4, 2.2	Ряд 1, место 7 → «Такого места нет.»
4	Повторять в while (true), выход по 0 через break	1.5	0 → «Хорошего сеанса!»

Это пригодится в Проекте 2: проверка, что ход игрока не выходит за поле.

Заготовка

Program.cs

```
char[][] hall = new char[4][];
hall[0] = new char[6];
// TODO: создай ряды 2–4: 8, 10 и 10 мест
// TODO: заполни все места точками '.' вложенными циклами

// TODO: выведи зал: "Ряд 1: . . . . . . "
while (true)
{
    // TODO: спроси ряд; 0 — выход (break)
    // TODO: спроси место; номера с 1, индексы с 0
    // TODO: место есть в ЭТОМ ряду? Свободно? Поставь 'X'
    // TODO: после брони снова выведи зал
}
Console.WriteLine("Хорошего сеанса!");
```

Вывод зала пока повторится дважды. В теме 2.4 научимся убирать такие повторы.

Объясни каждую строку своего кода — это часть сдачи.

Готово, если…

- Зал выводится четырьмя рядами по 6, 8, 10 и 10 мест.
- Ряд 1, место 7 → «Такого места нет.» Ряд 5 и буквы — то же.
- Ряд 2, место 3 второй раз → «Место занято, выбери другое.»
- Зал обходится вложенными циклами до hall[i].Length.

★ Касса

Ряды 1–2 — 300 ₽, ряды 3–4 — 450 ₽. После выхода выведи свободные места и выручку.

Вывод в консоли

```
Свободно: 33, выручка: 300 ₽
```

★★ Компания друзей

Вводишь число друзей — программа находит столько свободных мест подряд в одном ряду и бронирует их.

Вывод в консоли

```
Сколько вас? 3   <- ввёл пользователь
Ряд 1, места 1-3.
```

Стрелка — пометка для тебя, в программе её нет.

Если всё готово: игра «Жизнь»

Поле bool[,] 8 × 8: клетка живая или пустая. Каждое поколение считается по правилам:

- живая клетка с 2 или 3 живыми соседями выживает;
- пустая клетка ровно с 3 соседями оживает;
- остальные клетки становятся пустыми.

Выведи 5 поколений одно под другим: # — живая, . — пустая. Начни с фигуры «глайдер» и посмотри, как она ползёт.

Вывод в консоли

```
Поколение 1:
. # . . . . . .
. . # . . . . .
# # # . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
```

Итоги

Теперь я умею...

- хранить таблицу или игровое поле в двумерном массиве
- обходить таблицу вложенными циклами и узнавать её размеры
- считать суммы и средние по строкам и по столбцам
- хранить строки разной длины в ступенчатом массиве
- выбирать между таблицей и ступенчатым массивом